claude-windows / fa4dc4eb-4705-47fd-b044-4f017b586b15

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\fa4dc4eb-4705-47fd-b044-4f017b586b15\subagents\workflows\wf_b05b902c-a7c\agent-abc073728142f8540.jsonl`
- SHA-256: `29a0845d75a576ab469402cc177185b862e643dd8fe915e195e293c3522525f9`
- Source modified: `2026-06-19T16:29:39+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b05b902c-a7c`
- session_id: `fa4dc4eb-4705-47fd-b044-4f017b586b15`

Transcript

1. user (2026-06-19T16:25:21.915Z)

```
You are measuring the prominent-court shuttle gate's quality win on ONE multicourt badminton
golden clip, against human golden labels. Work in the repo root; run Python via Bash. Be
numerically exact ? do NOT estimate.

DATA
- Golden rallies: json.load(open("output/Badminton_BXH_2_golden_features.json"))["gts"] -> list
of [start,end] seconds.
- Manifest output/human_lovo_manifest.json: the entry with name==Badminton_BXH_2 gives
trajectory (path), fps, frame_width (=1920 for these; so frame_height=1080).

STEP 1 baseline (ungated) local rallies:
  from backend.eval.rally_seg_eval import windows_for
  from backend.eval.windowing import SERVED
  video={"name":NAME,"trajectory":TRAJ,"fps":FPS,"frame_width":FW,"gts":GTS}
  baseline = windows_for(video, SERVED)

STEP 2 derive a normalized (0-1) prominent-court polygon, RECALL-SAFE:
  from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
  points = TrackNetRunner.parse_trajectory_csv(TRAJ)   # each has .visible, .x, .y, .frame
  fh = 1080.0
  Inspect the 2D distribution of visible (x/FW, y/FH). Multicourt clips can have multiple
shuttle clusters (courts); the PROMINENT court = the dominant/foreground cluster (most shuttle
mass; foreground tends lower/larger in frame). Draw a polygon (quad or bounding box in
normalized coords) around the prominent cluster, EXCLUDING any clearly-separated secondary
(adjacent) cluster.
  VALIDATE recall-safety: for each golden rally compute its shuttle MEDIAN (x_norm,y_norm) over
frames in [s*FPS, e*FPS]; your polygon MUST contain >= 0.85 of golden-rally medians
(point_in_polygon from backend.pipeline.detectors.fusion_features). If < 0.85 widen it and re-
check; iterate. Record pct_golden_in_polygon. If the clip has NO clear secondary cluster to
exclude (courts overlap fully), say so ? a near-full-frame polygon is then a near no-op (an
honest null).

STEP 3 gate + score (numerically exact):
  from backend.pipeline.detectors.court_gate import gate_intervals_to_court
  gated = gate_intervals_to_court(baseline, points, polygon, FPS, FW, fh, min_in_court_frac=0.5)
  from backend.eval.metrics import score, match_intervals
  from backend.eval.tolerance_metrics import match_1to1
  b = score(baseline, GTS, 0.5); g = score(gated, GTS, 0.5)   # .precision/.recall/.f1
  base_tolf1 = match_1to1(baseline, GTS)[3]; gated_tolf1 = match_1to1(gated, GTS)[3]
  Of the DROPPED rallies (baseline minus gated), count how many were baseline FALSE POSITIVES: a
baseline pred is a FP if it is NOT in match_intervals(baseline, GTS, 0.5).matches as a matched
pred index.
  seg_ratio = n_pred / n_gt.
```

STEP 4 return the structured row. The win is REAL only if precision/F1 RISE with recall ~FLAT (you cut FPs, not real rallies). If recall drops materially, or the clip barely over-fires, or courts don't separate -> say so honestly (a null/negative result is a valid, valuable finding). pct_golden_in_polygon < 0.85 means your polygon is unsafe ? fix it before reporting.

YOUR CLIP: Badminton_BXH_2

2. assistant (2026-06-19T16:25:24.212Z)

I'll start by ramping up on the repo state and the relevant code, then run the analysis.

3. user (2026-06-19T16:25:27.206Z)

Already up to date.

4. user (2026-06-19T16:25:27.501Z)

```
19june.db
19june_extract.log
19june_extract_rerun.log
19june_local_vs_gemini.md
19june_work
Badminton_BXH_2.rallies.csv
Badminton_BXH_2_golden_features.json
Boxhill_Doubles.rallies.csv
Boxhill_Doubles_golden_features.json
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
GX010128.rallies.csv
GX010128_golden_features.json
GX010137.rallies.csv
GX010137_golden_features.json
GX010141.rallies.csv
GX010141_golden_features.json
GX020094.rallies.csv
GX020094_golden_features.json
GX030094.rallies.csv
GX030094_golden_features.json
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
REAL_FOOTAGE_VALIDATION_REPORT.md
_phase1_regress.log
_phase1_suite.log
_phase1_suite_fixed.log
_staleness_fixes.json
_suite_courtgate.log
_work19
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
---
output/Badminton_BXH_2_golden_features.json
output/human_lovo_manifest.json
```

5. assistant (2026-06-19T16:25:30.449Z)

Let me explore the relevant modules and data files.

6. user (2026-06-19T16:25:33.164Z)

```
keys: ['name', 'fps', 'frame_width', 'candidates', 'gts']
gts: [[0.413, 14.164], [22.666, 26.164], [36.665, 41.658], [49.907, 53.157], [62.906, 66.406],
[79.407, 86.907], [95.907, 98.407], [106.656, 108.907], [113.157, 116.157], [125.16, 127.907],
[133.656, 136.909], [143.246, 153.746], [164.746, 168.246], [174.497, 181.997], [191.746,
200.747], [210.747, 217.498], [222.997, 228.746], [235.497, 243.996], [258.497, 263.747],
[270.496, 278.496], [286.995, 290.746], [298.747, 308.248], [319.248, 323.497], [331.496,
334.893], [341.189, 345.19], [353.689, 365.939], [378.688, 381.685], [390.438, 395.189],
[471.327, 475.328], [482.826, 488.827], [495.606, 498.605], [505.853, 509.353], [516.353,
524.104], [535.352, 543.604], [551.352, 556.103], [565.103, 573.852], [582.353, 585.354],
[590.353, 598.102], [609.101, 612.353], [620.103, 628.351], [670.019, 675.268], [691.767,
696.517], [705.019, 707.518], [715.017, 722.518]]
n_gts: 44
```

7. user (2026-06-19T16:25:35.500Z)

```
list len 15
{
  "name": "Badminton_BXH_2",
  "fps": 59.94,
  "frame_width": 1920.0,
  "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Badminton_BXH_2.rallies.csv",
  "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv"
}
```

8. assistant (2026-06-19T16:25:38.357Z)

I have the manifest entry. Let me check the trajectory file exists and read the relevant code modules.

9. user (2026-06-19T16:25:40.616Z)

```
-rw-r--r-- 1 avidu 197609 1549493 Jun 17 15:02 C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv
---head---
Frame,Visibility,X,Y
0,1,822.7199935913086,531.8129539489746
1,1,823.1193351745605,533.6355400085449
2,1,824.5069313049316,536.7862129211426
3,0,-inf,-inf
```

10. user (2026-06-19T16:25:40.783Z)

```
1       """Prominent-court shuttle gate ? multicourt G2 (see
``docs/PROMINENT_COURT_DETECTION.md``).
2
3       In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
on a
4       background / adjacent court becomes a false rally on the filmed match. This gate re-
defines the
5       shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
a visible
6       shuttle point whose normalised position is outside the prominent-court polygon is marked
NOT
7       visible, so the velocity windower never opens a rally there.
8
9       Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10      ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
visibility, never
11      set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
```

```
    ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
    unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.

    The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
    ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
a point up
    to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
    admitted, so a high clear over the prominent court is not clipped.
    """
    from __future__ import annotations

    from dataclasses import replace
    from typing import Any, List, Optional, Sequence, Tuple

    from backend.pipeline.detectors.fusion_features import point_in_polygon

    Polygon = Sequence[Tuple[float, float]]


    def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
        """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?

        The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
        finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
        over the prominent court, admit it."""
        if point_in_polygon((nx, ny), polygon):
            return True
        return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)


    def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
                          fps: float, frame_width: float, frame_height: float,
                          height_pad: float = 0.0) -> float:
        """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.

        ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
        visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
        leave that judgement to the windower)."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return 1.0
        fs, fe = interval[0] * fps, interval[1] * fps
        n = inside = 0
        for p in points:
            if getattr(p, "visible", False) and fs <= p.frame <= fe:
                n += 1
                if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
                    inside += 1
        return 1.0 if n == 0 else inside / n


    def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
                                polygon: Optional[Polygon], fps: float,
                                frame_width: float, frame_height: float,
                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
                                ) -> List[Tuple[float, float]]:
```

```
66          """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68          Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69          occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70          window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
71          point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
72          so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
73          intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75              return intervals
76          return [iv for iv in intervals
77                  if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                  >= min_in_court_frac]
79
80
81      def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                               frame_width: float, frame_height: float,
83                               height_pad: float = 0.0) -> List[Any]:
84          """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85          polygon (expanded upward by ``height_pad``).
86
87          No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88          are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89          its points are never mutated)."""
90          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91              return points
92          out: List[Any] = []
93          for p in points:
94              if getattr(p, "visible", False):
95                  nx, ny = p.x / frame_width, p.y / frame_height
96                  if not admitted(nx, ny, polygon, height_pad):
97                      p = replace(p, visible=False)
98              out.append(p)
99          return out
100
```

11. assistant (2026-06-19T16:25:42.666Z)

Now let me read the key helper modules to understand exact signatures.

12. user (2026-06-19T16:25:47.067Z)

```
imports OK
SERVED= WindowingPreset(name='served', velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=2.0, max_window_duration=6.0, hard_cap=False, inactivity_end=1.5,
inactivity_rally_thresh=0.2, inactivity_min_density=0.3, blob_fallback=False, blob_factor=4.0)
windows_for sig: (video: 'Dict[str, Any]', preset: 'WindowingPreset', min_crossings: 'int' = 0)
-> 'List[Interval]'
parse sig: (csv_win_path: str) ->
List[backend.pipeline.detectors.tracknet_runner.TrajectoryPoint]
point_in_polygon sig: (point: 'Tuple[float, float]', polygon: 'Sequence[Tuple[float, float]]')
-> 'bool'
match_intervals sig: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]],
iou_threshold: float = 0.5) -> backend.eval.metrics.MatchResult
```

```
match_1to1 sig: (preds: 'List[Interval]', gts: 'List[Interval]', tau_start: 'float' = 2.0,
tau_end: 'float' = 1.5) -> 'Tuple[List[Match], float, float, float]'
score sig: (preds: List[Tuple[float, float]], gts: List[Tuple[float, float]], iou_threshold:
float = 0.5, match_result: Optional[backend.eval.metrics.MatchResult] = None) ->
backend.eval.metrics.Score
```

13. assistant (2026-06-19T16:25:48.903Z)

Now let me get baseline windows and inspect the trajectory distribution.

14. user (2026-06-19T16:25:53.254Z)

```
n baseline: 78
  [0.017, 31.331]
  [34.268, 41.525]
  [43.560, 46.079]
  [50.184, 56.473]
  [57.975, 60.761]
  [61.411, 66.216]
  [69.286, 71.405]
  [71.939, 85.769]
  [91.508, 94.378]
  [95.162, 99.149]
  [102.486, 108.509]
  [111.879, 119.536]
  [120.020, 122.306]
  [124.007, 128.095]
  [132.699, 137.354]
  [142.843, 153.487]
  [156.590, 159.109]
  [163.297, 169.036]
  [170.187, 172.890]
  [173.807, 178.228]
  [178.912, 182.449]
  [188.789, 201.752]
  [202.769, 206.573]
  [210.093, 215.516]
  [216.583, 228.295]
  [230.230, 233.350]
  [233.851, 238.622]
  [239.523, 247.748]
  [258.208, 263.981]
  [269.203, 278.862]
  [284.368, 291.458]
  [295.362, 308.976]
  [311.895, 314.164]
  [314.948, 317.885]
  [320.687, 322.689]
  [331.481, 334.985]
  [340.574, 345.395]
  [346.246, 348.298]
  [353.070, 365.449]
  [366.600, 372.356]
  [373.090, 375.526]
  [378.545, 382.199]
  [382.833, 388.338]
  [390.307, 394.645]
  [404.004, 408.192]
  [410.961, 417.651]
  [419.319, 422.039]
  [424.308, 430.597]
  [433.851, 441.758]
  [443.243, 446.213]
  [461.545, 465.816]
  [467.835, 475.509]
```

```
  [477.294, 480.998]
  [482.816, 487.554]
  [488.922, 494.645]
  [496.096, 498.248]
  [499.183, 524.191]
  [527.044, 529.613]
  [535.485, 542.359]
  [542.926, 547.614]
  [548.982, 560.544]
  [565.716, 581.465]
  [586.737, 589.173]
  [591.959, 600.667]
  [604.938, 612.913]
  [614.181, 617.534]
  [619.336, 634.401]
  [635.385, 638.021]
  [642.075, 647.981]
  [652.052, 656.723]
  [657.407, 662.045]
  [663.964, 670.571]
  [671.371, 675.492]
  [684.952, 690.724]
  [692.676, 697.030]
  [704.755, 714.097]
  [714.631, 722.890]
  [725.325, 727.961]
```

15. assistant (2026-06-19T16:25:57.166Z)

Baseline has 78 predictions vs 44 golden rallies ? heavy over-firing (seg_ratio ~1.77), consistent with multicourt false rallies. Now let me parse the trajectory and analyze the shuttle distribution to find clusters.

16. user (2026-06-19T16:26:02.212Z)

```
n points: 43695
attrs: ['frame', 'visible', 'x', 'y']
n visible: 29373
n visible finite: 29373
x range: 0.003 .. 0.969
y range: 0.000 .. 0.940
```

17. assistant (2026-06-19T16:26:05.141Z)

Points span nearly the full frame. Let me build a 2D histogram to see the cluster structure and distinguish prominent vs adjacent courts.

18. user (2026-06-19T16:26:09.803Z)

```
Exit code 1
  File "<string>", line 16
    print('y\x  '+''.join('%4d'%i for i in range(G)))
          ^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 1-2: truncated
\xXX escape
```

19. assistant (2026-06-19T16:26:13.246Z)

Let me write this to a script file to avoid shell escaping issues.

20. assistant (2026-06-19T16:26:19.900Z)

```
import math
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

```
TRAJ = 'C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv'
pts = TrackNetRunner.parse_trajectory_csv(TRAJ)
FW = 1920.0
FH = 1080.0
vis = [(p.x / FW, p.y / FH) for p in pts
       if getattr(p, 'visible', False) and math.isfinite(p.x) and math.isfinite(p.y)]
N = len(vis)

G = 20
grid = [[0] * G for _ in range(G)]
for x, y in vis:
    gx = min(G - 1, int(x * G))
    gy = min(G - 1, int(y * G))
    grid[gy][gx] += 1

header = 'y/x  ' + ''.join('%4d' % i for i in range(G))
print(header)
for gy in range(G):
    row = grid[gy]
    print('%4.2f ' % ((gy + 0.5) / G) + ''.join('%4d' % (v * 1000 // N) for v in row))
print('total visible N=', N, '(cells show per-mille of total)')
```

column marginal (x) and row marginal (y)

```
colsum = [sum(grid[gy][gx] for gy in range(G)) for gx in range(G)]
rowsum = [sum(grid[gy]) for gy in range(G)]
print('\nX marginal (per-mille):')
print('  ' + ''.join('%4.2f' % ((gx + 0.5) / G) for gx in range(G)))
print('  ' + ''.join('%4d' % (c * 1000 // N) for c in colsum))
print('\nY marginal (per-mille):')
for gy in range(G):
    print('  y=%4.2f : %4d  %s' % ((gy + 0.5) / G, rowsum[gy] * 1000 // N, '#' * (rowsum[gy] *
200 // N)))
```

21. user (2026-06-19T16:26:19.942Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\bxh2_hist.py (file state is current in your context ? no need to Read it back)
```

22. user (2026-06-19T16:26:23.480Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_hist.py", line 2, in
<module>
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
ModuleNotFoundError: No module named 'backend'
```

23. assistant (2026-06-19T16:26:27.030Z)

```
The cwd resets between Bash calls. Let me set PYTHONPATH / run from repo root explicitly.
```

24. user (2026-06-19T16:26:27.781Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_hist.py", line 2, in
<module>
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
ModuleNotFoundError: No module named 'backend'
```

25. assistant (2026-06-19T16:26:31.502Z)

The earlier inline `python -c` worked because the harness ran from repo root, but the script run isn't picking up `backend`. The inline commands worked, so the cwd is correct but a `.pth`/conftest mechanism may add the path. Let me check how the inline calls resolved and add the repo root to sys.path explicitly in the script.

26. user (2026-06-19T16:26:34.532Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_hist.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. user (2026-06-19T16:26:37.963Z)

```
y/x     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19
0.03    0   0   0   0   0   0   0   0   0   1   2   1   0   0   0   0   0   0   0   0
0.07    0   1   0   0   0   0   1   1   2   3   4   3   1   1   0   0   0   0   0   0
0.12    0   1   0   0   0   0   1   1   4   3   3   2   0   0   1   0   0   0   0   0
0.17    1   4   0   0   0   0   1   2   2   5   3   2   1   0   1   0   0   0   0   0
0.23    2   7   2   0   0   1   1   4  10   6   1   0   0   0   0   0   0   0   0   0
0.28    3   9   7   0   0   1   3   6  12   4   1   1   0   0   0   0   0   0   0   0
0.33    5  10   7   0   0   1   4  10  10 415   5   1   0   0   0   0   0   0   0   0
0.38    3  10   7   0   0   1   4  12  13   8   6   2   0   0   0   0   0   0   0   0
0.42    1   6   4   0   0   2   9  16  23  15   6   3   2   0   0   0   0   0   0   0
0.47    0   2   2   0   0   4  13   8  18   9   5   2   0   0   0   0   0   0   0   0
0.53    0   1   1   0   1   1   2   3   3   3   2   1   1   2   0   0   0   0   0   0
0.57    0   0   1   0   1   1   2   3   3   1   9   0   0   2   0   0   0   0   0   0
0.62    0   0   0   0   0   0   5   5  11   4   1   0   0   0   0   0   0   0   0   0
0.68    0   2   0   0   0   0   0   2   0   0   0   0   0   0   0   0   0   0   0   0
0.72    0   0   0   0   0   0   0   0   0   0   0   0   2   0   0   0   0   0   0   0
0.78    0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0.82    0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0.88    0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0.93    0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0.97    0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
total visible N= 29373 (cells show per-mille of total)

X marginal (per-mille):
  0.030.070.120.170.230.280.330.380.420.470.530.570.620.680.720.780.820.880.930.97
    22  61  35   3   4  17  54  79 118 483  55  23  13  12   5   2   2   2   0   0

Y marginal (per-mille):
  y=0.03 :   10  ##
  y=0.07 :   23  ####
  y=0.12 :   22  ####
  y=0.17 :   28  #####
  y=0.23 :   44  ########
  y=0.28 :   54  ##########
  y=0.33 :  475
##################################################################################################
  y=0.38 :   72  ##############
  y=0.42 :   93  ##################
  y=0.47 :   71  ##############
  y=0.53 :   27  #####
  y=0.57 :   30  ######
  y=0.62 :   30  ######
  y=0.68 :    7  #
  y=0.72 :    5  #
  y=0.78 :    0
  y=0.82 :    0
  y=0.88 :    0
  y=0.93 :    0
  y=0.97 :    0
```

28. assistant (2026-06-19T16:26:45.451Z)

There's a huge spike at cell (gx=9, gy=6) = x?[0.45,0.50], y?[0.30,0.35] with 415 per-mille (41.5% of all points). That single cell dominates ? likely a "parking" position where the tracker sits on a stationary/default point, not real shuttle motion. Let me look more carefully. The structure shows:

- A dominant lower-left/central blob: x roughly [0.0, 0.55], y roughly [0.15, 0.65] ? this is one court.
- The mass is mostly in the left-center of the frame. There's very little on the right side (x > 0.6).

This looks like there may be one prominent court (left/center) and the right side is largely empty. But the 41.5% spike in one cell is suspicious ? it could be a static detection artifact. Let me investigate that cell and check whether golden rallies map to it, plus look at how golden-rally medians distribute. The key question: is there a clearly separated secondary cluster to exclude?

29. assistant (2026-06-19T16:26:55.681Z)

```
import math
import os
import sys
import json
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

TRAJ = 'C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv'
FW = 1920.0
FH = 1080.0
FPS = 59.94
pts = TrackNetRunner.parse_trajectory_csv(TRAJ)
g = json.load(open('output/Badminton_BXH_2_golden_features.json'))
GTS = g['gts']
```

Investigate the dominant cell x in [0.45,0.50], y in [0.30,0.35]

```
hot = [p for p in pts if getattr(p, 'visible', False) and math.isfinite(p.x)
       and 0.45 <= p.x / FW < 0.50 and 0.30 <= p.y / FH < 0.35]
print('hot-cell count:', len(hot))
```

exact x,y modes

```
from collections import Counter
c = Counter((round(p.x, 1), round(p.y, 1)) for p in hot)
print('top exact (x,y) rounded:', c.most_common(5))
```

raw values

```
cr = Counter((round(p.x, 3), round(p.y, 3)) for p in hot)
print('top raw (x,y):', cr.most_common(5))
```

Per-golden-rally shuttle MEDIAN (x_norm, y_norm)

```
print('\nGolden-rally medians (x_norm, y_norm):')
medians = []
for (s, e) in GTS:
    fs, fe = s * FPS, e * FPS
    xs = [p.x / FW for p in pts if getattr(p, 'visible', False) and math.isfinite(p.x) and fs <=
p.frame <= fe]
    ys = [p.y / FH for p in pts if getattr(p, 'visible', False) and math.isfinite(p.y) and fs <=
p.frame <= fe]
    if xs:
        xs.sort(); ys.sort()
        mx = xs[len(xs) // 2]; my = ys[len(ys) // 2]
    else:
        mx = my = float('nan')
    medians.append((mx, my))
    print('  [%7.3f,%7.3f] n=%4d  med=(%.3f, %.3f)' % (s, e, len(xs), mx, my))
```

```
mxs = [m[0] for m in medians if math.isfinite(m[0])]
mys = [m[1] for m in medians if math.isfinite(m[1])]
print('\ngolden-median x range: %.3f .. %.3f' % (min(mxs), max(mxs)))
print('golden-median y range: %.3f .. %.3f' % (min(mys), max(mys)))
```

30. user (2026-06-19T16:26:55.691Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\bxh2_clusters.py (file state is current in your context ? no need to Read it
back)
```

31. user (2026-06-19T16:27:02.002Z)

```
hot-cell count: 12216
top exact (x,y) rounded: [((896.2, 371.2), 833), ((896.2, 375.0), 358), ((896.2, 373.0), 258),
((896.2, 373.1), 256), ((895.4, 374.9), 242)]
top raw (x,y): [((896.25, 371.25), 831), ((896.25, 375.0), 251), ((896.25, 373.049), 11),
((896.25, 373.046), 9), ((896.25, 373.008), 9)]

Golden-rally medians (x_norm, y_norm):
  [  0.413, 14.164] n= 780  med=(0.467, 0.347)
  [ 22.666, 26.164] n= 198  med=(0.356, 0.345)
  [ 36.665, 41.658] n= 275  med=(0.426, 0.348)
  [ 49.907, 53.157] n= 140  med=(0.469, 0.347)
  [ 62.906, 66.406] n= 199  med=(0.469, 0.347)
  [ 79.407, 86.907] n= 410  med=(0.457, 0.347)
  [ 95.907, 98.407] n= 126  med=(0.638, 0.289)
  [106.656,108.907] n= 114  med=(0.436, 0.347)
  [113.157,116.157] n= 172  med=(0.468, 0.299)
  [125.160,127.907] n= 143  med=(0.454, 0.347)
  [133.656,136.909] n= 194  med=(0.098, 0.363)
  [143.246,153.746] n= 555  med=(0.434, 0.346)
  [164.746,168.246] n= 209  med=(0.115, 0.389)
  [174.497,181.997] n= 365  med=(0.399, 0.346)
  [191.746,200.747] n= 486  med=(0.434, 0.339)
  [210.747,217.498] n= 317  med=(0.467, 0.347)
  [222.997,228.746] n= 327  med=(0.463, 0.347)
  [235.497,243.996] n= 432  med=(0.424, 0.345)
  [258.497,263.747] n= 311  med=(0.469, 0.291)
  [270.496,278.496] n= 402  med=(0.415, 0.346)
  [286.995,290.746] n= 196  med=(0.469, 0.346)
  [298.747,308.248] n= 540  med=(0.452, 0.379)
  [319.248,323.497] n= 168  med=(0.466, 0.347)
  [331.496,334.893] n= 191  med=(0.469, 0.345)
  [341.189,345.190] n= 222  med=(0.358, 0.347)
  [353.689,365.939] n= 656  med=(0.454, 0.346)
  [378.688,381.685] n= 131  med=(0.456, 0.327)
  [390.438,395.189] n= 243  med=(0.467, 0.347)
  [471.327,475.328] n= 224  med=(0.469, 0.347)
  [482.826,488.827] n= 282  med=(0.395, 0.246)
  [495.606,498.605] n= 105  med=(0.476, 0.344)
  [505.853,509.353] n= 195  med=(0.465, 0.396)
  [516.353,524.104] n= 448  med=(0.430, 0.365)
  [535.352,543.604] n= 432  med=(0.438, 0.285)
  [551.352,556.103] n= 270  med=(0.426, 0.347)
  [565.103,573.852] n= 478  med=(0.413, 0.333)
  [582.353,585.354] n= 133  med=(0.424, 0.344)
  [590.353,598.102] n= 372  med=(0.466, 0.399)
  [609.101,612.353] n= 155  med=(0.399, 0.349)
  [620.103,628.351] n= 437  med=(0.471, 0.368)
  [670.019,675.268] n= 218  med=(0.432, 0.398)
  [691.767,696.517] n= 212  med=(0.465, 0.426)
  [705.019,707.518] n= 121  med=(0.402, 0.429)
  [715.017,722.518] n= 370  med=(0.442, 0.344)
```

```
golden-median x range: 0.098 .. 0.638
golden-median y range: 0.246 .. 0.429
```

32. assistant (2026-06-19T16:27:11.901Z)

Very informative. Key findings:

1. **The dominant cell is a tracker artifact**: 12,216 points (~42% of all visible) sit at essentially one fixed pixel (896.25, 371.25) = norm (0.467, 0.347). 831 hits at exactly that pixel ? this is a static "parked" detection (the tracker's default/idle position), not real shuttle motion. This explains the 41.5% spike. Critically, this artifact sits at the CENTER of the prominent court, so it doesn't threaten recall.

2. **Golden-rally medians cluster tightly in the prominent court**: x?[0.098, 0.638], y?[0.246, 0.429]. All 44 golden medians are in this band. Notably:
   - Two golden rallies have x-medians far left (~0.10): the rallies at [133.656,136.909] (x=0.098) and [164.746,168.246] (x=0.115). These are real and must stay inside.
   - One golden median reaches x=0.638 (rally at [95.907,98.407]).
   - All y-medians are between 0.246 and 0.429 (upper-middle of frame).

The prominent court occupies x?[~0.0, 0.65], y?[~0.15, 0.65]. The right portion of the frame (x>0.65) and bottom (y>0.65) have almost no shuttle mass. The question: **is there a clearly-separated secondary cluster (adjacent court) to exclude?** Looking at the histogram, the mass is one connected blob ? there is no clearly separated second cluster. Let me verify the baseline false-positive structure to see WHERE the over-firing predictions' shuttle lives, and whether a polygon can cut them.

33. assistant (2026-06-19T16:27:23.736Z)

```
import math
import os
import sys
import json
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.eval.rally_seg_eval import windows_for
from backend.eval.windowing import SERVED
from backend.eval.metrics import match_intervals

TRAJ = 'C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv'
FW = 1920.0
FH = 1080.0
FPS = 59.94
pts = TrackNetRunner.parse_trajectory_csv(TRAJ)
g = json.load(open('output/Badminton_BXH_2_golden_features.json'))
GTS = g['gts']
video = {'name': 'Badminton_BXH_2', 'trajectory': TRAJ, 'fps': FPS, 'frame_width': FW, 'gts':
GTS}
baseline = windows_for(video, SERVED)

mr = match_intervals(baseline, GTS, 0.5)
# matched pred indices
matched_pred = set(m[0] for m in mr.matches)
print('n baseline:', len(baseline), 'matched preds (TP):', len(matched_pred),
      'FP:', len(baseline) - len(matched_pred))


def med(iv):
    s, e = iv
    fs, fe = s * FPS, e * FPS
    xs = [p.x / FW for p in pts if getattr(p, 'visible', False) and math.isfinite(p.x) and fs <=
p.frame <= fe]
    ys = [p.y / FH for p in pts if getattr(p, 'visible', False) and math.isfinite(p.y) and fs <=
```

```
p.frame <= fe]
    if not xs:
        return (float('nan'), float('nan'), 0)
    xs.sort(); ys.sort()
    return (xs[len(xs) // 2], ys[len(ys) // 2], len(xs))


print('\nBaseline FALSE POSITIVES (pred not matched to any golden), with shuttle median:')
fp_meds = []
for i, iv in enumerate(baseline):
    if i not in matched_pred:
        mx, my, n = med(iv)
        fp_meds.append((mx, my))
        print('  FP [%7.3f,%7.3f] n=%4d med=(%.3f,%.3f)' % (iv[0], iv[1], n, mx, my))

print('\nBaseline TRUE POSITIVES shuttle median x-range:')
tp_meds = []
for i, iv in enumerate(baseline):
    if i in matched_pred:
        mx, my, n = med(iv)
        tp_meds.append((mx, my))
fxs = [m[0] for m in fp_meds if math.isfinite(m[0])]
fys = [m[1] for m in fp_meds if math.isfinite(m[1])]
txs = [m[0] for m in tp_meds if math.isfinite(m[0])]
tys = [m[1] for m in tp_meds if math.isfinite(m[1])]
if fxs:
    print('FP median x: %.3f..%.3f  y: %.3f..%.3f  (n=%d)' % (min(fxs), max(fxs), min(fys),
max(fys), len(fxs)))
print('TP median x: %.3f..%.3f  y: %.3f..%.3f  (n=%d)' % (min(txs), max(txs), min(tys),
max(tys), len(txs)))
```

34. user (2026-06-19T16:27:23.747Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\bxh2_fp.py (file state is current in your context ? no need to Read it back)
```

35. user (2026-06-19T16:27:30.287Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_fp.py", line 23, in
<module>
    matched_pred = set(m[0] for m in mr.matches)
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_fp.py", line 23, in
<genexpr>
    matched_pred = set(m[0] for m in mr.matches)
                       ~^^^
TypeError: 'Match' object is not subscriptable
```

36. assistant (2026-06-19T16:27:34.130Z)

Let me check the Match / MatchResult structure.

37. user (2026-06-19T16:27:36.030Z)

```
@dataclass
class MatchResult:
    """Outcome of matching predictions to ground truth at a given IoU threshold."""
    matches: List[Match] = field(default_factory=list)
    unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
    unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives (misses)

---Match---
['Interval', 'List', 'Match', 'MatchResult', 'Optional', 'Score', 'Tuple', 'dataclass', 'field',
```

'interval_iou', 'match_intervals', 'pr_curve', 'score']

38. assistant (2026-06-19T16:27:39.752Z)

The `Match` has named fields. Let me find the field name for the pred index.
`unmatched_pred_indices` directly gives FPs, which is what I need.

39. user (2026-06-19T16:27:42.936Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\bxh2_fp.py has been updated
successfully. (file state is current in your context ? no need to Read it back)

40. user (2026-06-19T16:27:53.612Z)

n baseline: 78 matched preds (TP): 27 FP: 51

Baseline FALSE POSITIVES (pred not matched to any golden), with shuttle median:
  FP [0.017, 31.331] n=1649 med=(0.466,0.347)
  FP [43.560, 46.079] n= 140 med=(0.094,0.328)
  FP [50.184, 56.473] n= 269 med=(0.469,0.347)
  FP [57.975, 60.761] n= 154 med=(0.467,0.347)
  FP [69.286, 71.405] n= 104 med=(0.469,0.347)
  FP [71.939, 85.769] n= 680 med=(0.467,0.347)
  FP [91.508, 94.378] n= 114 med=(0.467,0.346)
  FP [102.486,108.509] n= 317 med=(0.427,0.346)
  FP [111.879,119.536] n= 427 med=(0.465,0.346)
  FP [120.020,122.306] n= 132 med=(0.469,0.347)
  FP [156.590,159.109] n= 121 med=(0.090,0.347)
  FP [170.187,172.890] n= 112 med=(0.468,0.347)
  FP [173.807,178.228] n= 197 med=(0.410,0.340)
  FP [178.912,182.449] n= 188 med=(0.397,0.353)
  FP [202.769,206.573] n= 143 med=(0.414,0.346)
  FP [216.583,228.295] n= 523 med=(0.467,0.347)
  FP [230.230,233.350] n= 154 med=(0.469,0.347)
  FP [233.851,238.622] n= 218 med=(0.467,0.346)
  FP [239.523,247.748] n= 409 med=(0.466,0.346)
  FP [311.895,314.164] n= 94 med=(0.415,0.384)
  FP [314.948,317.885] n= 117 med=(0.468,0.347)
  FP [320.687,322.689] n= 96 med=(0.372,0.373)
  FP [346.246,348.298] n= 80 med=(0.468,0.347)
  FP [366.600,372.356] n= 184 med=(0.468,0.347)
  FP [373.090,375.526] n= 90 med=(0.469,0.347)
  FP [382.833,388.338] n= 269 med=(0.469,0.346)
  FP [404.004,408.192] n= 157 med=(0.469,0.347)
  FP [410.961,417.651] n= 303 med=(0.468,0.346)
  FP [419.319,422.039] n= 31 med=(0.467,0.346)
  FP [424.308,430.597] n= 274 med=(0.469,0.347)
  FP [433.851,441.758] n= 318 med=(0.468,0.347)
  FP [443.243,446.213] n= 119 med=(0.060,0.338)
  FP [461.545,465.816] n= 141 med=(0.466,0.347)
  FP [477.294,480.998] n= 145 med=(0.129,0.320)
  FP [488.922,494.645] n= 177 med=(0.364,0.347)
  FP [499.183,524.191] n=1256 med=(0.467,0.347)
  FP [527.044,529.613] n= 112 med=(0.497,0.620)
  FP [542.926,547.614] n= 244 med=(0.106,0.398)
  FP [548.982,560.544] n= 562 med=(0.466,0.349)
  FP [565.716,581.465] n= 782 med=(0.466,0.347)
  FP [586.737,589.173] n= 44 med=(0.466,0.410)
  FP [604.938,612.913] n= 398 med=(0.466,0.347)
  FP [614.181,617.534] n= 158 med=(0.467,0.348)
  FP [635.385,638.021] n= 94 med=(0.171,0.349)
  FP [642.075,647.981] n= 205 med=(0.461,0.201)
  FP [652.052,656.723] n= 188 med=(0.469,0.347)
  FP [657.407,662.045] n= 212 med=(0.467,0.348)

```
  FP [663.964,670.571] n= 320 med=(0.103,0.347)
  FP [684.952,690.724] n=  99 med=(0.262,0.352)
  FP [704.755,714.097] n= 504 med=(0.371,0.347)
  FP [725.325,727.961] n=  59 med=(0.466,0.347)

Baseline TRUE POSITIVES shuttle median x-range:
FP median x: 0.060..0.497  y: 0.201..0.620  (n=51)
TP median x: 0.098..0.484  y: 0.241..0.448  (n=27)
```

41. assistant (2026-06-19T16:28:05.337Z)

This is the critical finding. The baseline false positives have shuttle medians sitting at **the exact same location as the true positives** ? the prominent court center (0.467, 0.347). The FPs are NOT on a separate adjacent court; they are mostly:

- The dominant tracker-artifact location (0.467, 0.347) ? the "parked" idle detection.
- Real prominent-court locations (x ~0.06-0.49, y ~0.20-0.62) that overlap entirely with TPs.

The FP median x range (0.060?0.497) and TP median x range (0.098?0.484) are essentially identical. The FP y range extends slightly further (0.201?0.620 vs 0.241?0.448), but only 2 FPs sit outside the TP envelope (one at y=0.620, one at y=0.201).

There is no clearly-separated secondary cluster (adjacent court) to exclude. The over-firing here is NOT from a background court whose shuttle lives in a different image region ? it's the windower opening rallies on the static-tracker artifact and on genuine prominent-court motion during non-rally periods. A spatial court polygon cannot separate these because the FPs and TPs are spatially co-located.

Let me confirm this is an honest null by actually running the gate with a recall-safe polygon and measuring. I'll derive the tightest recall-safe polygon (must contain ?85% of golden medians) and check what it can possibly drop.

42. assistant (2026-06-19T16:28:25.386Z)

```
import math
import os
import sys
import json
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.eval.rally_seg_eval import windows_for
from backend.eval.windowing import SERVED
from backend.pipeline.detectors.court_gate import gate_intervals_to_court
from backend.pipeline.detectors.fusion_features import point_in_polygon
from backend.eval.metrics import score, match_intervals
from backend.eval.tolerance_metrics import match_1to1

TRAJ = 'C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv'
FW = 1920.0
FH = 1080.0
FPS = 59.94
pts = TrackNetRunner.parse_trajectory_csv(TRAJ)
g = json.load(open('output/Badminton_BXH_2_golden_features.json'))
GTS = g['gts']
video = {'name': 'Badminton_BXH_2', 'trajectory': TRAJ, 'fps': FPS, 'frame_width': FW, 'gts':
GTS}
baseline = windows_for(video, SERVED)


def gmed(s, e):
    fs, fe = s * FPS, e * FPS
    xs = [p.x / FW for p in pts if getattr(p, 'visible', False) and math.isfinite(p.x) and fs <=
p.frame <= fe]
    ys = [p.y / FH for p in pts if getattr(p, 'visible', False) and math.isfinite(p.y) and fs <=
```

```
p.frame <= fe]
    if not xs:
        return None
    xs.sort(); ys.sort()
    return (xs[len(xs) // 2], ys[len(ys) // 2])


golden_medians = [gmed(s, e) for (s, e) in GTS]
golden_medians = [m for m in golden_medians if m is not None]
NG = len(golden_medians)


def pct_in(poly):
    inside = sum(1 for m in golden_medians if point_in_polygon(m, poly))
    return inside / NG


def evaluate(poly, label, min_frac=0.5):
    pct = pct_in(poly)
    gated = gate_intervals_to_court(baseline, pts, poly, FPS, FW, FH,
min_in_court_frac=min_frac)
    b = score(baseline, GTS, 0.5)
    gg = score(gated, GTS, 0.5)
    btol = match_1to1(baseline, GTS)[3]
    gtol = match_1to1(gated, GTS)[3]
    # dropped intervals
    gated_set = set(gated)
    dropped = [iv for iv in baseline if iv not in gated_set]
    mr = match_intervals(baseline, GTS, 0.5)
    fp_idx = set(mr.unmatched_pred_indices)
    base_index = {iv: i for i, iv in enumerate(baseline)}
    dropped_fp = sum(1 for iv in dropped if base_index[iv] in fp_idx)
    print('=== %s  min_frac=%.2f ===' % (label, min_frac))
    print('  pct_golden_in_polygon: %.4f' % pct)
    print('  baseline: n=%d P=%.4f R=%.4f F1=%.4f tolF1=%.4f' % (len(baseline), b.precision,
b.recall, b.f1, btol))
    print('  gated   : n=%d P=%.4f R=%.4f F1=%.4f tolF1=%.4f' % (len(gated), gg.precision,
gg.recall, gg.f1, gtol))
    print('  dropped=%d  dropped_were_FP=%d' % (len(dropped), dropped_fp))
    print('  deltaP=%+.4f deltaR=%+.4f deltaF1=%+.4f' % (gg.precision - b.precision, gg.recall -
b.recall, gg.f1 - b.f1))
    return dict(poly=poly, label=label, min_frac=min_frac, pct=pct, b=b, gg=gg, btol=btol,
gtol=gtol,
                dropped=len(dropped), dropped_fp=dropped_fp, ng=NG)
```

Candidate polygon: tight bbox around golden medians + margin, but recall-safe.

```
mxs = [m[0] for m in golden_medians]
mys = [m[1] for m in golden_medians]
print('golden median x %.3f..%.3f  y %.3f..%.3f, NG=%d' % (min(mxs), max(mxs), min(mys),
max(mys), NG))
```

Polygon A: bounding box of prominent court (recall-safe by construction: contains all

```
pad = 0.02
A = [(max(0.0, min(mxs) - pad), max(0.0, min(mys) - pad)),
     (min(1.0, max(mxs) + pad), max(0.0, min(mys) - pad)),
     (min(1.0, max(mxs) + pad), min(1.0, max(mys) + pad)),
     (max(0.0, min(mxs) - pad), min(1.0, max(mys) + pad))]
print('Polygon A bbox:', [(round(x, 3), round(y, 3)) for x, y in A])

resA50 = evaluate(A, 'A=tight-bbox', 0.5)
resA50b = evaluate(A, 'A=tight-bbox', 0.7)
```

Polygon B: also cap the right side at 0.65 and bottom at 0.65 to exclude the empty righ

```
B = [(0.0, 0.10), (0.66, 0.10), (0.66, 0.66), (0.0, 0.66)]
resB = evaluate(B, 'B=left-court-quad x<=0.66 y<=0.66', 0.5)
```

43. user (2026-06-19T16:28:25.500Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\bxh2_gate.py (file state is current in your context ? no need to Read it back)
```

44. user (2026-06-19T16:28:35.726Z)

```
golden median x 0.098..0.638  y 0.246..0.429, NG=44
Polygon A bbox: [(0.078, 0.226), (0.658, 0.226), (0.658, 0.449), (0.078, 0.449)]
=== A=tight-bbox  min_frac=0.50 ===
  pct_golden_in_polygon: 1.0000
  baseline: n=78 P=0.3462 R=0.6136 F1=0.4426 tolF1=0.3279
  gated   : n=65 P=0.3231 R=0.4773 F1=0.3853 tolF1=0.2936
  dropped=13  dropped_were_FP=7
  deltaP=-0.0231 deltaR=-0.1364 deltaF1=-0.0573
=== A=tight-bbox  min_frac=0.70 ===
  pct_golden_in_polygon: 1.0000
  baseline: n=78 P=0.3462 R=0.6136 F1=0.4426 tolF1=0.3279
  gated   : n=32 P=0.1250 R=0.0909 F1=0.1053 tolF1=0.0789
  dropped=46  dropped_were_FP=23
  deltaP=-0.2212 deltaR=-0.5227 deltaF1=-0.3374
=== B=left-court-quad x<=0.66 y<=0.66  min_frac=0.50 ===
  pct_golden_in_polygon: 1.0000
  baseline: n=78 P=0.3462 R=0.6136 F1=0.4426 tolF1=0.3279
  gated   : n=78 P=0.3462 R=0.6136 F1=0.4426 tolF1=0.3279
  dropped=0  dropped_were_FP=0
  deltaP=+0.0000 deltaR=+0.0000 deltaF1=+0.0000
```

45. assistant (2026-06-19T16:28:46.984Z)

This confirms the honest null/negative result. Two cases:

1. **Polygon B** (the recall-safe left-court quad x?0.66, y?0.66, which contains 100% of golden medians): it is a **near no-op** ? drops 0 intervals. Because all real shuttle mass (TPs AND FPs) lives within x?0.66, y?0.66, the polygon doesn't cut anything. The "adjacent court" simply doesn't produce a spatially separable cluster here.

2. **Polygon A** (tight bbox of golden medians, x?[0.078,0.658], y?[0.226,0.449]): even at min_frac=0.5 it **hurts** ? drops 13 intervals, but only 7 were FPs (the other 6 were TPs, real rallies). Precision DROPS (-0.023), recall DROPS hard (-0.136), F1 drops (-0.057). At min_frac=0.7 it's catastrophic. The tight bbox isn't a legitimate "court" ? it's just a tight crop on golden medians, and the gate's per-window in-court fraction (which uses ALL visible points in a window, not the median) drops real rallies whose shuttle excursions (clears, lobs going to y>0.449 or x>0.658) push them below 50% in-court. So it cuts real rallies, not a separate court.

The proper recall-safe polygon (B), which is the honest representation of "the prominent court region," is a no-op. Let me verify B is genuinely the right recall-safe choice and finalize. Let me also confirm B contains the slightly-out FP at y=0.620 to be thorough, and report the no-op B as the result (it's the recall-safe one). Let me double-check the exact baseline/gated scores and the dropped-FP count under the reported polygon B.

46. assistant (2026-06-19T16:29:01.005Z)

```
import math
import os
import sys
import json
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

```
from backend.eval.rally_seg_eval import windows_for
from backend.eval.windowing import SERVED
from backend.pipeline.detectors.court_gate import gate_intervals_to_court
from backend.pipeline.detectors.fusion_features import point_in_polygon
from backend.eval.metrics import score, match_intervals
from backend.eval.tolerance_metrics import match_1to1

TRAJ = 'C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv'
FW = 1920.0
FH = 1080.0
FPS = 59.94
pts = TrackNetRunner.parse_trajectory_csv(TRAJ)
g = json.load(open('output/Badminton_BXH_2_golden_features.json'))
GTS = g['gts']
video = {'name': 'Badminton_BXH_2', 'trajectory': TRAJ, 'fps': FPS, 'frame_width': FW, 'gts':
GTS}
baseline = windows_for(video, SERVED)


def gmed(s, e):
    fs, fe = s * FPS, e * FPS
    xs = [p.x / FW for p in pts if getattr(p, 'visible', False) and math.isfinite(p.x) and fs <=
p.frame <= fe]
    ys = [p.y / FH for p in pts if getattr(p, 'visible', False) and math.isfinite(p.y) and fs <=
p.frame <= fe]
    if not xs:
        return None
    xs.sort(); ys.sort()
    return (xs[len(xs) // 2], ys[len(ys) // 2])


golden_medians = [m for m in (gmed(s, e) for (s, e) in GTS) if m is not None]
NG = len(golden_medians)
```

RECALL-SAFE prominent-court polygon: foreground court spans the left/center frame and bottom (y>0.66) of the frame carry essentially no shuttle mass. This quad contains golden-rally medians.

```
POLY = [(0.0, 0.08), (0.68, 0.08), (0.68, 0.68), (0.0, 0.68)]

pct = sum(1 for m in golden_medians if point_in_polygon(m, POLY)) / NG

gated = gate_intervals_to_court(baseline, pts, POLY, FPS, FW, FH, min_in_court_frac=0.5)

b = score(baseline, GTS, 0.5)
gg = score(gated, GTS, 0.5)
btol = match_1to1(baseline, GTS)[3]
gtol = match_1to1(gated, GTS)[3]

mr = match_intervals(baseline, GTS, 0.5)
fp_idx = set(mr.unmatched_pred_indices)
gated_set = set(gated)
dropped = [iv for iv in baseline if iv not in gated_set]
base_index = {iv: i for i, iv in enumerate(baseline)}
dropped_fp = sum(1 for iv in dropped if base_index[iv] in fp_idx)

out = dict(
    name='Badminton_BXH_2',
    n_gt=len(GTS),
    pct_golden_in_polygon=round(pct, 6),
    polygon=POLY,
    base_n_pred=len(baseline),
    base_precision=round(b.precision, 6),
```

```
    base_recall=round(b.recall, 6),
    base_f1=round(b.f1, 6),
    base_tolf1=round(btol, 6),
    gated_n_pred=len(gated),
    gated_precision=round(gg.precision, 6),
    gated_recall=round(gg.recall, 6),
    gated_f1=round(gg.f1, 6),
    gated_tolf1=round(gtol, 6),
    n_dropped=len(dropped),
    n_dropped_were_fp=dropped_fp,
    delta_precision=round(gg.precision - b.precision, 6),
    delta_recall=round(gg.recall - b.recall, 6),
    delta_f1=round(gg.f1 - b.f1, 6),
    seg_ratio=round(len(baseline) / len(GTS), 4),
)
print(json.dumps(out, indent=2))
```

47. user (2026-06-19T16:29:01.036Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\bxh2_final.py (file state is current in your context ? no need to Read it back)
```

48. user (2026-06-19T16:29:08.526Z)


```
{
  "name": "Badminton_BXH_2",
  "n_gt": 44,
  "pct_golden_in_polygon": 1.0,
  "polygon": [
    [
      0.0,
      0.08
    ],
    [
      0.68,
      0.08
    ],
    [
      0.68,
      0.68
    ],
    [
      0.0,
      0.68
    ]
  ],
  "base_n_pred": 78,
  "base_precision": 0.346154,
  "base_recall": 0.613636,
  "base_f1": 0.442623,
  "base_tolf1": 0.327869,
  "gated_n_pred": 78,
  "gated_precision": 0.346154,
  "gated_recall": 0.613636,
  "gated_f1": 0.442623,
  "gated_tolf1": 0.327869,
  "n_dropped": 0,
  "n_dropped_were_fp": 0,
  "delta_precision": 0.0,
  "delta_recall": 0.0,
  "delta_f1": 0.0,
  "seg_ratio": 1.7727
}
```

49. assistant (2026-06-19T16:29:15.170Z)

The result is confirmed and numerically exact. The recall-safe prominent-court polygon (containing 100% of golden-rally medians) is a **complete no-op** ? it drops 0 of the 78 baseline predictions, so every metric is unchanged. This is an honest null result.

The reason is clear from the diagnostics: in this clip the baseline over-fires heavily (78 preds vs 44 golden, seg_ratio 1.77, precision 0.346), but the false positives are **spatially co-located with the true positives** in the prominent court ? they are NOT a separable adjacent-court cluster. ~42% of all visible shuttle points are a static tracker artifact parked at the prominent-court center (0.467, 0.347), and the FP shuttle medians (x 0.060?0.497, y 0.201?0.620) sit on top of the TP medians (x 0.098?0.484, y 0.241?0.448). There is no clearly-separated secondary court cluster to exclude, so any recall-safe polygon admits everything. Tightening it to actually cut intervals immediately destroys recall (drops real rallies, not a background court).

Let me clean up the scratch files.

50. user (2026-06-19T16:29:16.753Z)

cleaned

51. assistant (2026-06-19T16:29:21.518Z)

I have all numerically-exact results. Returning the structured row.

52. user (2026-06-19T16:29:31.034Z)

Structured output provided successfully

53. assistant (2026-06-19T16:29:39.555Z)

Returned the structured row.

Summary: For Badminton_BXH_2 the prominent-court shuttle gate produces an **honest null** ? zero quality change (?P=?R=?F1=0.0). The recall-safe polygon (containing 100% of the 44 golden-rally medians) drops 0 of 78 baseline predictions because this clip has no spatially-separable adjacent-court cluster: the false positives are co-located with the true rallies in the prominent court, and ~42% of all visible shuttle points are a static tracker artifact parked at court-center (0.467, 0.347). The baseline heavily over-fires (seg_ratio 1.77, precision 0.346), but that over-firing is temporal/idle-tracker noise, not a background court ? so a spatial court gate cannot help here, and any polygon tight enough to cut intervals destroys recall instead.

Relevant files:
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\court_gate.py (gate under test)
- C:\Users\avidu\Projects\badminton-highlight-indexer\output\Badminton_BXH_2_golden_features.json (golden labels)
- C:\Users\avidu\Projects\badminton-highlight-indexer\output\human_lovo_manifest.json (trajectory/fps/dims)
- Trajectory: C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\Badminton_BXH_2_june15_traj.csv

(Scratch analysis scripts were created under scratch/ and removed after the run; scratch/ is gitignored.)